Comparative Study of Traditional Machine Learning and Transformer Models for Spam Classification

Technical report | Experimental results from Google Colab | Binary Spam/Ham classification

Abstract

This study compares two traditional text-classification pipelines, TF-IDF with Logistic Regression and TF-IDF with Multinomial Naive Bayes, against a fine-tuned DistilBERT transformer. A supplied spam-message dataset contained 5,573 records and two fields; after cleaning and duplicate removal, 5,157 records remained (4,516 Ham; 641 Spam). A stratified 70/15/15 training/validation/test protocol was used for all models. On 774 held-out test messages, DistilBERT achieved 98.84% accuracy and 95.19% Spam F1-score, compared with 97.80% and 90.29% for Naive Bayes, and 96.51% and 83.64% for Logistic Regression. DistilBERT improved Spam recall but required substantially more training time.

1. Introduction and objective

Spam classification is a binary natural-language-processing task in which unwanted or fraudulent messages are identified without unnecessarily blocking legitimate messages. Traditional machine learning relies on engineered features such as term frequency-inverse document frequency (TF-IDF). Transformer architectures instead learn contextual representations through pretrained language modeling. This experiment assesses classification quality and training cost using the same dataset split and common evaluation measures.

2. Dataset and exploratory analysis

The dataset was obtained from the Kaggle Spam Email Classification dataset specified in the assignment (ashfakyeafi/spam-email-classification). The two columns were Category (Ham or Spam) and Message (message text). Missing records and duplicate rows were removed, labels were standardized, and empty messages were excluded. Labels were encoded as Ham = 0 and Spam = 1.

Measure	Observed value
Original dataset	5,573 rows; 2 columns
Cleaned dataset	5,157 messages
Ham	4,516 (87.57%)
Spam	641 (12.43%)
Class balance	Imbalanced toward Ham

Because Spam represents only 12.43% of the cleaned data, overall accuracy alone can be misleading. Spam-class precision, recall and F1-score were therefore emphasized. The stratified split maintained approximately the same class proportions in training, validation and testing.

3. Methodology and experimental setup

3.1 Data partitioning and leakage control

A random seed of 42 was used with a stratified 70% training, 15% validation and 15% test split. The validation and test sets each contained 774 messages, including 678 Ham and 96 Spam. All models were trained on the same training partition. TF-IDF vocabulary fitting occurred within a scikit-learn pipeline on training data only. The test set was reserved for final evaluation; DistilBERT checkpoint selection used validation Spam F1-score.

3.2 Traditional machine learning

Both traditional classifiers used lowercase TF-IDF features with word unigrams and bigrams, minimum document frequency of 2 and sublinear term-frequency scaling. This retains informative terms and short phrases while excluding extremely rare terms. Logistic Regression was trained with a maximum of 1,000 iterations; Multinomial Naive Bayes used additive smoothing with alpha = 0.5. Both methods are efficient sparse-text baselines, but they do not directly model contextual meaning in the way a transformer does.

3.3 Transformer-based classification

The pretrained distilbert-base-uncased model was fine-tuned for two-class sequence classification in PyTorch. Its tokenizer generated input IDs and attention masks with truncation/padding to 128 tokens. Training used a manually written PyTorch loop: forward pass, cross-entropy loss, backward propagation, gradient clipping, optimizer update and learning-rate scheduler update. The high-level Hugging Face Trainer API was not used. All model parameters were trainable (no layer freezing).

Setting	DistilBERT configuration
Compute	Google Colab; NVIDIA Tesla T4 GPU
Batch size	16
Epochs	3
Learning rate	2 × 10^-5
Optimizer / regularization	AdamW; weight decay 0.01; gradient clipping 1.0
Scheduler	Linear decay; 10% warmup
Checkpoint selection	Highest validation Spam F1-score

3.4 Evaluation metrics

All models were assessed using Accuracy, Precision, Recall, F1-score and a confusion matrix. For precision, recall and F1-score, Spam (label 1) was treated as the positive class. Precision quantifies the correctness of Spam alerts, whereas recall measures the fraction of actual Spam detected. F1-score balances these two criteria. Wall-clock training time was recorded to compare computational cost.

4. Experimental results

Table 3 reports final held-out test results, not validation metrics. DistilBERT ranked highest in accuracy, Spam recall and Spam F1-score. The traditional methods had perfect observed Spam precision on this test partition, while DistilBERT traded a small number of false positives for substantially fewer missed Spam messages.

Model	Accuracy	Spam precision	Spam recall	Spam F1
Logistic Regression	0.9651	1.0000	0.7188	0.8364
Naive Bayes	0.9780	1.0000	0.8229	0.9029
DistilBERT	0.9884	0.9780	0.9271	0.9519

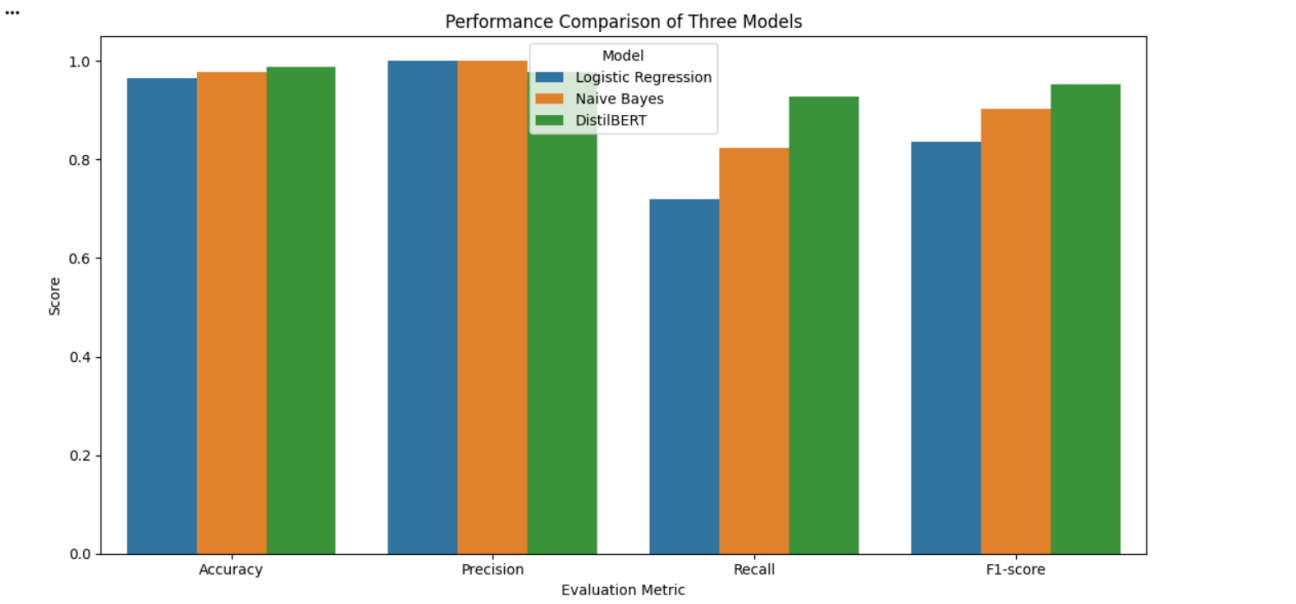


Figure 1. Test-set performance comparison (user-generated Colab plot).

DistilBERT correctly classified 765 of 774 messages. Its test confusion matrix contains 676 true Ham, 89 true Spam, 2 Ham incorrectly flagged as Spam and 7 Spam incorrectly classified as Ham. Compared with Naive Bayes, its Spam recall increased by 10.42 percentage points and Spam F1-score by 4.90 percentage points.

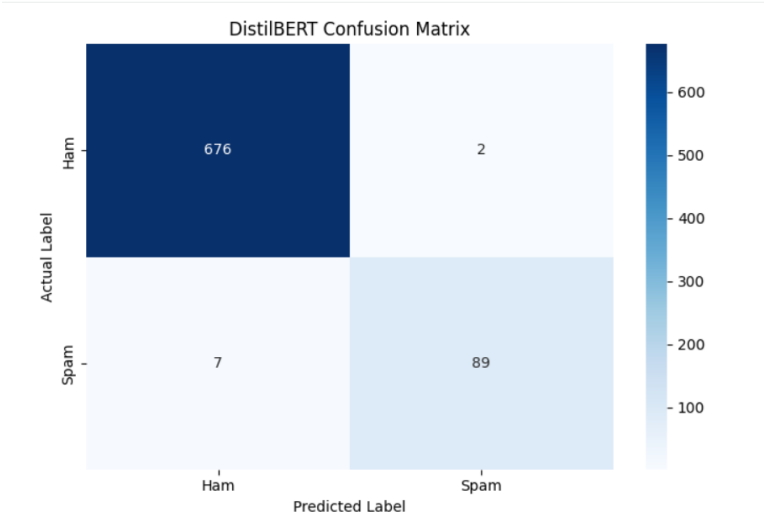


Figure 2. DistilBERT test confusion matrix: rows are actual classes, columns are predictions.

5. Training behavior and computational cost

The DistilBERT training loss decreased strongly over three epochs, whereas validation loss increased slightly. This divergence suggests some overfitting even though validation classification scores remained high. The checkpoint with the best validation Spam F1-score was saved for final test evaluation.

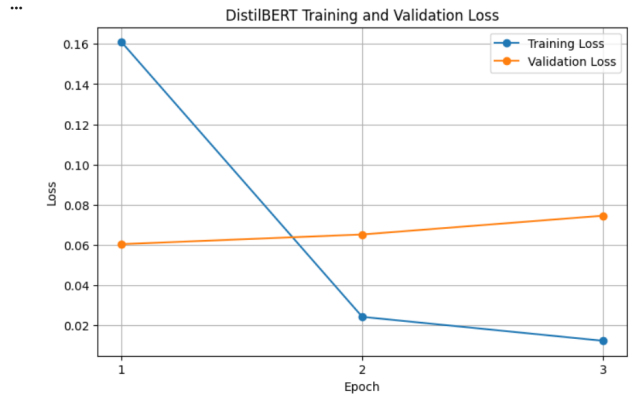


Figure 3. Training and validation loss over three epochs.

Traditional models were much faster to train: approximately 1.15 s for Logistic Regression and 0.36 s for Naive Bayes. The provided training-time chart places DistilBERT near 133 s (approximate graph reading; the exact Colab timer output was not supplied). DistilBERT test inference over 774 messages took 3.31 s. These figures reflect CPU-based classical models and GPU-based transformer training, so they should not be interpreted as a controlled same-hardware speed benchmark.

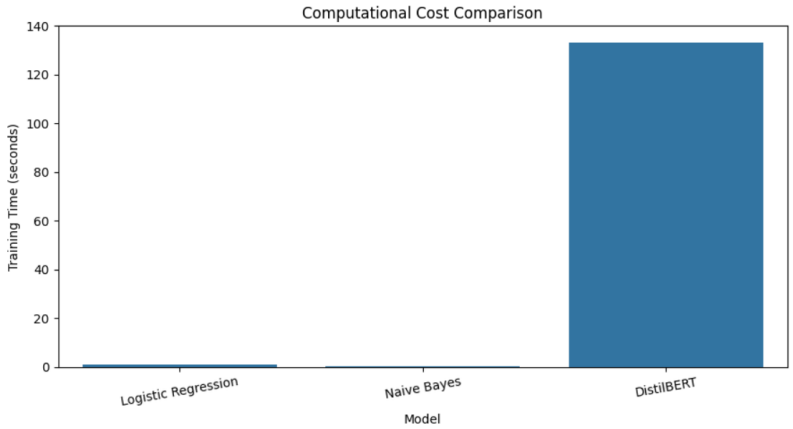


Figure 4. Training-time comparison from the experiment.

6. Limitations and comparative discussion

The dataset is relatively small and class-imbalanced, and its message style may not represent modern enterprise email traffic or adversarial spam. Duplicate removal reduced possible repetition, but generalization to new domains remains untested. The 128-token truncation may discard information in long messages. A single train/test split and one random seed do not establish statistical significance; further runs and external validation would improve confidence. No broad hyperparameter search was performed. The transformer showed signs of overfitting in validation loss, and the timing comparison uses different hardware types.

7. Conclusion

All three methods classified Spam and Ham effectively. Naive Bayes was the strongest classical baseline, reaching 97.80% test accuracy and 90.29% Spam F1-score with very low training cost. DistilBERT achieved the best detection performance, with 98.84% accuracy, 92.71% Spam recall and 95.19% Spam F1-score. For applications where reducing missed Spam is critical and GPU resources are available, DistilBERT is

attractive; for fast, resource-constrained deployments, Naive Bayes remains a practical option.

References: Kaggle, “Spam Email Classification,” https://www.kaggle.com/datasets/ashfakyeafi/spam-email-classification. Hugging Face, “DistilBERT model documentation,” https://huggingface.co/docs/transformers/en/model_doc/distilbert.